API Development Standards

Document ID	ENG-002
Department	Engineering
Document type	Standard
Owner	Director of Platform Engineering
Version	1.2
Status	APPROVED
Effective / last reviewed	2026-01-01
Audience	Engineering staff
Related documents	ENG-001, ENG-010, CS-002, CS-006

Summary. Sets the design and operating standards for Northstar REST APIs, covering URL versioning, pagination, rate limits, error format, authentication, timeouts, idempotency, deprecation notice and Sunset headers, breaking-change rules, documentation, testing and review gates.

SYNTHETIC DOCUMENT - Northstar Systems is a fictional company. This document exists for development, testing and demonstration only and is not real legal, employment, security or compliance advice.

1. Purpose and Scope

This standard defines how APIs are designed, built, versioned, documented and retired at Northstar Systems Private Limited. It applies to every public API exposed to customers through Nexus Connect, to the APIs of the Northstar Nexus Platform and Nexus Insight, and to internal service-to-service APIs. Public APIs follow every rule in this document. Internal APIs follow the same rules unless a section says otherwise.

Consistent APIs reduce integration effort for customers, lower the number of support tickets that reach Customer Success, and let teams reuse client libraries and tooling. The standard is owned by the Director of Platform Engineering. Pull requests that change an API are reviewed under ENG-001, and API-related support codes and customer behaviour are described in CS-002 and CS-006. Deployment of API changes follows ENG-010 and the standard release process.

2. Definitions

- **Public API**: an API that customers or partners call from outside Northstar networks, published through Nexus Connect.
- **Internal API**: an API used between Northstar services only.
- **Tenant**: a customer account in the Northstar Nexus Platform. Rate limits and quotas are applied per tenant.
- **Major version**: the version number in the URL (for example `/v1/`). A new major version may contain breaking changes.
- **Breaking change**: a change that can make a correctly written existing client fail, such as removing a field, renaming a field, changing a type, making an optional field required, or changing the meaning of a status code.
- **Deprecation**: the announced plan to remove an API or version. **Sunset**: the date when the API stops working.
- **Idempotency key**: a client-generated unique value that lets a request be retried safely.

- **Cursor**: an opaque token that marks the position in a list of results.

3. Roles and Responsibilities

- **API owner (team)**: designs the API, keeps the specification current, supports customers and meets deprecation commitments.
- **Director of Platform Engineering**: owns this standard and approves exceptions.
- **API reviewers (Architecture Review Board)**: review new public APIs and new major versions. The Architecture Review Board (ARB) meets every second Thursday at 16:00 IST.
- **Security Champion**: reviews authentication, authorisation and data exposure in API changes.
- **Technical Writer**: reviews reference documentation for public APIs.
- **Customer Success and Support**: relay customer feedback, communicate deprecations and use the error codes for troubleshooting.

4. Design Principles

1 APIs are REST over HTTPS, with JSON request and response bodies, unless an ADR approves another style.
2 Resources are nouns in plural form (for example `/v1/workspaces/{id}/pipelines`). Actions that do not map to a resource are modelled as sub-resources or custom actions and need ARB review.
3 Use standard HTTP methods: GET for reads, POST to create, PUT to replace, PATCH to partially update and DELETE to remove. GET must never change state.
4 Field names use lower camelCase and timestamps use ISO 8601 in UTC.
5 Be strict in what you send and tolerant in what you accept: clients must ignore unknown fields, so adding an optional response field is not a breaking change.
6 Design for backward compatibility from the first release, since public APIs live for a long time.

5. Versioning

APIs use URL versioning with the major version at the start of the path, for example `/v1/pipelines`. Only the major version appears in the URL. Within a major version, only backwards-compatible changes are allowed. Breaking changes are allowed only in a new major version, which runs alongside the previous one until the previous version is sunset.

Examples of non-breaking changes: adding an optional request field, adding a response field, adding a new endpoint, adding a new enum value where the documentation says clients must tolerate unknown values. Examples of breaking changes: removing or renaming a field, changing a field type, changing the default page size, tightening validation so that a previously valid request is rejected, and changing authentication requirements.

Release notes for every API change are published to the developer portal on the day of release.

6. Pagination, Filtering and Sorting

List endpoints must use cursor pagination. The default page size is 50 items and the maximum page size is 200. Requests above 200 are either capped at 200 or rejected with a 400 error, as stated in the API specification; the choice must be consistent across the API. The response includes a `nextCursor` value, which is omitted on the last page. Clients treat cursors as opaque and must not construct them. Cursors remain valid for at least 24 hours.

Filtering uses query parameters named after the field (for example `status=active`). Sorting uses `sort=createdAt` or `sort=-createdAt` for descending. Unsupported filters return 400.

Example: A customer lists Pipelines in a Workspace without a limit. The API returns 50 Pipelines and a `nextCursor`. The customer asks for `limit=500`; the API caps at 200 and states in the response that the limit was adjusted. Both behaviours are documented in the reference.

7. Rate Limiting

The default rate limit is 1,000 requests per minute per tenant. When the limit is exceeded, the API returns HTTP 429 with a `Retry-After` header that gives the number of seconds to wait. The Nexus Platform surfaces this as NX-3010 (rate limit exceeded), and support guidance for customers is to use exponential backoff and honour `Retry-After`.

Every response includes rate limit headers that show the limit, the remaining requests and the reset time. Higher limits for a tenant may be agreed by Customer Success and approved by the Director of Platform Engineering. Limits are set per tenant, not per user or per token, so one noisy integration can affect others in the same tenant; API owners should recommend that customers separate high-volume jobs and use batch endpoints.

Internal APIs use the same mechanism, and teams should set limits that protect Tier-1 services from callers in lower tiers.

8. Error Handling

Errors use the problem-details format (the `application/problem+json` media type). Each error response includes `type`, `title`, `status`, `detail` and `instance`, and the Northstar extensions `requestId` and `code`. The `requestId` is required for support: customers should send it when they open a ticket. Error messages must be helpful and must not contain internal system names, stack traces, SQL or personal data.

Use these status codes consistently:

Status	Use
200	Successful read or update
201	Resource created, with a Location header
202	Accepted for asynchronous processing
204	Successful delete with no body
400	Malformed request or invalid parameter
401	Missing or invalid credentials (NX-1001 for an expired or invalid token)
403	Authenticated but not allowed
404	Resource not found
409	Conflict, for example a duplicate create
422	Valid syntax but a semantic validation failure
429	Rate limit exceeded
500	Unexpected server error (NX-5000)
503	Temporarily unavailable, with Retry-After

A 5xx response must never leak details. 500 and 503 events are logged with the requestId so that Support and Engineering can find the cause quickly.

9. Authentication and Authorisation

Service access to public APIs uses OAuth 2.0 client credentials. Clients get a client ID and secret for each Workspace and exchange them for short-lived access tokens. Tokens must be sent in the Authorization header, never in the URL. Secrets are shown once at creation, and customers should rotate them regularly. An expired or invalid token returns 401, which the platform reports as NX-1001; the customer fixes this by rotating the token in the Workspace settings.

Authorisation follows least privilege: each token carries scopes that limit which resources it can read or write, and the API checks the scope and the tenant on every request. An API must never return data from another tenant. Internal service-to-service calls use mutual TLS or signed tokens from NorthID-managed service identities. Service account credentials are rotated every 90 days and stored in NorthVault or the KMS-backed secret store.

All traffic uses TLS; TLS 1.2 is the minimum and TLS 1.3 is preferred. Public APIs are reachable only through the load balancer and web application firewall.

10. Timeouts, Retries and Idempotency

The default request timeout is 30 seconds. An operation that may take longer than 30 seconds must be asynchronous: the API returns 202 with a status URL that the client polls. Connectors in Nexus Pipelines have a default timeout of 30 seconds that can be raised to 120 seconds; a connector that exceeds it produces NX-2004 (connector timeout).

POST requests that create resources must accept an `Idempotency-Key` header. If the same key is sent again within 24 hours with the same body, the API returns the original response without creating a second resource. If the same key is sent with a different body, the API returns 409 or 422 as documented. PUT and DELETE are idempotent by definition and must be implemented that way. Clients should retry only idempotent requests, or POST requests with an idempotency key, using exponential backoff with jitter.

Example: A customer's integration creates an invoice record with a POST and the network drops before the response arrives. The client retries with the same Idempotency-Key. The API recognises the key and returns the original 201 response, so no duplicate invoice is created.

11. Deprecation and Sunset

A public API or version that will be removed must be announced with at least 12 months' notice before the Sunset date. The following steps apply:

1 The API owner raises a deprecation request to the Director of Platform Engineering with the reason, the replacement and the proposed Sunset date.
2 After approval, the API owner announces the deprecation in release notes, the developer portal and by email to affected customer contacts through Customer Success.
3 From the announcement date, every response from the deprecated API includes `Deprecation` and `Sunset` headers and a `Link` header pointing to the migration guide.
4 Usage is monitored in NorthWatch. Customer Success contacts tenants that are still calling the API at 6 months and 3 months before the Sunset date.

5 On the Sunset date, the API returns 410 Gone with a problem-details body that points to the replacement. No extension may be granted unless the Director of Platform Engineering approves it.

Internal APIs need at least 3 months' notice to dependent teams, tracked in a NorthDesk ticket.

Example: Version 1 of the Pipelines API is replaced by version 2. The deprecation is announced on 1 March 2026, so the earliest allowed Sunset date for v1 is 1 March 2027. The team sets 15 March 2027, and every v1 response from 1 March 2026 carries the Sunset header with that date.

12. Documentation, Testing and Review

Every API must have an OpenAPI specification stored in version control, with examples for each endpoint, error responses and authentication. The specification is the source of the reference documentation. Contract tests run in the NorthShip pipeline and fail the build when the implementation differs from the specification. The pipeline also runs a breaking-change check that compares the new specification with the last released one; a flagged breaking change fails the build unless it is in a new major version.

New public APIs and new major versions need ARB review before development finishes, and before any release the API owner completes the API checklist in NorthDesk: naming, pagination, errors, authentication, rate limits, idempotency, documentation, observability and security review. Code review follows ENG-001; security-sensitive changes need review by a Security Champion.

Each API must emit metrics for request rate, error rate and latency, and have a dashboard of the golden signals if it is Tier-1. Paging alerts must relate to customer-facing symptoms and link to a runbook.

12.1 Pre-release API checklist in detail

Before an API change is released, the API owner confirms the following and records the answers in the NorthDesk checklist ticket:

1 The resource names, field names and status codes follow sections 4 and 8.
2 List endpoints use cursor pagination with the default page size of 50 and the maximum of 200.
3 The rate limit headers are returned and the tenant limit of 1,000 requests per minute is enforced at the gateway.
4 Every POST that creates a resource accepts an idempotency key.
5 The error format is problem-details and includes the requestId; no internal names or personal data appear in messages.
6 The OpenAPI specification is updated, the contract tests pass, and the breaking-change check is clean or the change is in a new major version.
7 Authentication uses OAuth 2.0 client credentials with scopes, and tenant isolation tests pass.
8 Dashboards, alerts and a runbook exist, and the on-call team has been informed.
9 Release notes and the migration guide (if relevant) are ready for the developer portal.

12.2 Working with Customer Success

API changes can generate support tickets, so the API owner gives Customer Success at least 5 business days' notice of any customer-visible change, with a short summary, known limitations and the troubleshooting codes involved. Support uses the codes NX-1001 (authentication failure), NX-2004 (connector timeout), NX-3010 (rate limit exceeded), NX-4003 (schema mismatch in a Pipeline) and NX-5000 (internal error) when helping customers. If a new error code is added, the

API owner must give Support the cause, the fix and the information needed for a ticket, such as tenant ID, Workspace name, version, request ID and timestamp.

Example: A team adds a new validation rule to a Pipeline endpoint, which would make some previously valid payloads fail with 422. Because this tightens validation, it is a breaking change and cannot be shipped within v1. The team ships the rule in v2, adds a warning header to v1 responses and announces the difference in the release notes.

13. Exceptions

An API owner who cannot follow a rule must raise a written exception request to the Director of Platform Engineering with the reason, the risk and an expiry date, not longer than 12 months. Common approved exceptions include streaming endpoints that cannot use cursor pagination and legacy partner integrations that use a different authentication method. Exceptions for security rules, such as TLS versions or tenant isolation, need approval from the CISO delegate as well.

14. Frequently Asked Questions

What is the default and maximum page size? The default page size is 50 and the maximum is 200.

What happens when I exceed the rate limit? The API returns HTTP 429 with a Retry-After header; the default limit is 1,000 requests per minute per tenant.

How much notice do customers get for deprecations? At least 12 months, with a Sunset header on responses.

15. Related Documents

Code review rules are in ENG-001. Deployment practice is in ENG-010. Customer troubleshooting codes and support handling are in CS-002 and CS-006.

16. Revision History

Version	Date	Change
1.0	2025-03-10	Initial API standard
1.1	2025-09-22	Added idempotency rules and rate limit headers
1.2	2026-01-01	Aligned deprecation notice to 12 months and added breaking-change check